



**BERLIN SCHOOL OF
BUSINESS & INNOVATION**

Essay / Assignment Title:

Leveraging Machine Learning Approaches for Stroke Risk Prediction

Programme title:

Introduction to Artificial Intelligence export

Name:

Mahla Alimardani, Q1107537

Year:

2025

CONTENTS

Chapters	Pages
Introduction	4
Chapter One	
Exploratory Data Analysis	5
Chapter Two	
Data Preparation/ Model Training/ Model Evaluation and Visualization	13
Chapter Three	
Conclusion and Future Work	35
Conclusion	38
References	39

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters):

.....MAHLA ALIMARDANI.....

Date:09...../.....29.../.....2025..

INTRODUCTION

When blood cannot reach a part of brain, a stroke will happen. It can make it hard for a person to hardly speech or move and even recovery may take a long time ([NHS, n.d.](#)). Stroke is the second common cause of death globally that cause 5.5 million deaths in 2015 ([Daidone, Ferrantelli and Tuttolomondo, 2023](#)), also It has been found out that 90 percent of all strokes are due to modifiable risk factors, which include high blood pressure, obesity, physical inactivity, unhealthy diet as well as smoking. Thus, It is important to identify those people at high risk early to avert stroke and minimize complications. The latest developments that have been achieved in machine learning provide effective tools used in prediction of stroke risk based on analysis of complex trends in patient data. Through these computational methods, healthcare providers are able to introduce timely interventions and customize treatment plans and eventually enhance patient outcomes. The proposed project is based on the idea of using machine-learning models in order to predict the risk of stroke to offer a precise, data-driven method to assist with preventative healthcare.

The source files to all this project can be obtained through the following link.

<https://github.com/mahlaalimardani/13711371>

CHAPTER ONE

Exploratory Data Analysis

The major numeric variables will be age, avg_glucose_level, bmi, and the target variable will be stroke; the value of bmi has some missing values.

1. Target (stroke)

The dataset is highly imbalanced. Predicting class 1 is harder, and plain accuracy can be misleading.

2. Age

- Mean, median, min, and max for age is 43.2, 45, 0.08, 82.

Histogram and boxplot: large distribution throughout adulthood, median of 45, long tails. Age is probably a risk-causing factor that builds up. There is something strange with the very small minimum (newborn or data problem).

3. Avg_glucose_level

- Mean = 106, std = 45, min = 55, max = 272.

Histogram: multi-modal: there are 1 large population with the centre near 80-100 and the other one with the center near 190-210. This means refers to various health profiles (high glucose vs normal). There is a good possibility that this variable would be a strong predictor.

4. BMI

- BMI has Mean 28.9, SD 7.847, minimum value 10.3, maximum value 97.6, and some (NaN) values are missing.

Plot: skewed to the right having outliers. There are numerous individuals that are in overweight or obese ranges, they have extremes and they might correspond to uncommon events.

5. Categorical snapshot (of the head)

Variables are hypertension, heart disease, ever married, work type, Residence type, work status. These provide both lifestyle and medical history and are able to interact with age and glucose.

6. Overview

It is a binary classification dataset on a rare positive. Glucose has a high separation pattern, age indicates an increasing evidence in terms of risk, BMI is skewed and partially missing. Based on the visuals, we would anticipate by default that higher age, higher glucose, and higher BMI are more correlated with stroke and would be substantiated through modeling.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import matplotlib.pyplot as plt
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
from google.colab import files
uploaded= files.upload()
import pandas as pd
df= pd.read_csv("healthcare-dataset-stroke-data.csv") # Data.csv = file's name
df.head()
```

healthcare-dataset-stroke-data.csv(text/csv) - 322083 bytes, last modified: 9/27/2025 - 100% done
Saving healthcare-dataset-stroke-data.csv to healthcare-dataset-stroke-data.csv

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

```
import matplotlib.pyplot as plt
counts_target=df["stroke"].value_counts(dropna=False)
print(counts_target)

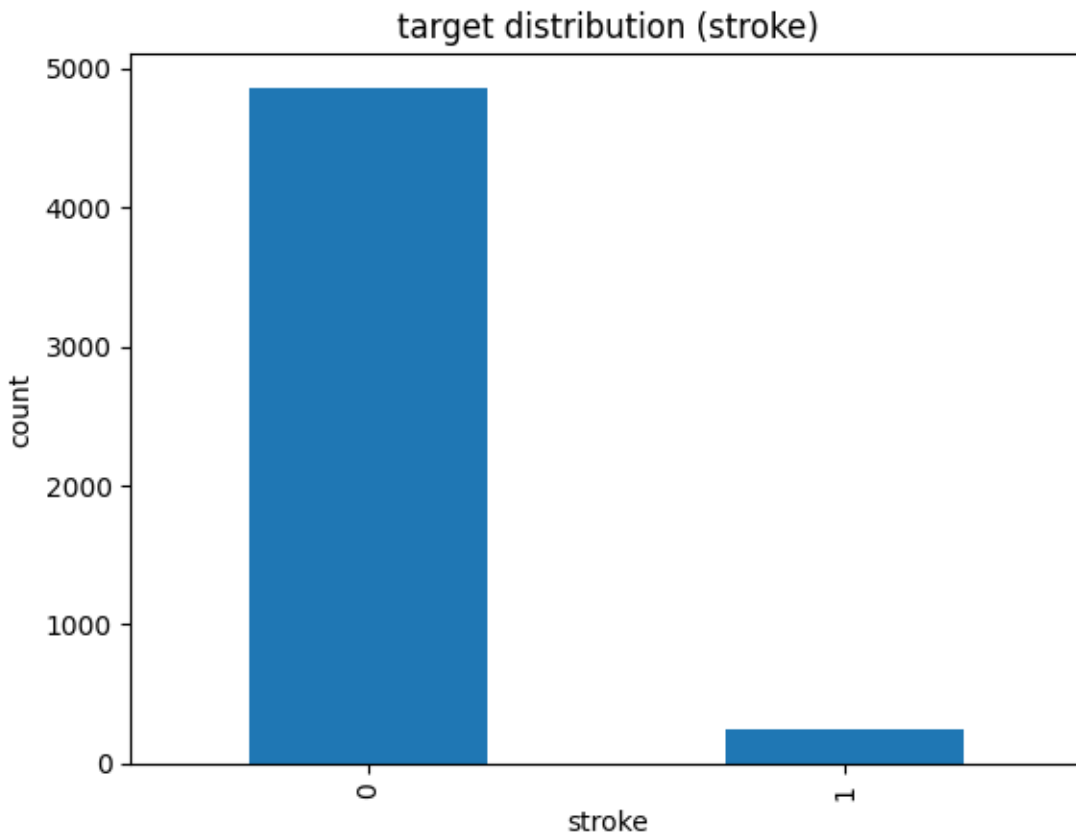
target_ratio=df["stroke"].value_counts(dropna=False,normalize=True)
print(target_ratio)

print((target_ratio*100).round(2))

counts_target.plot(kind="bar",title="target distribution (stroke)");plt.xlabel("stroke");plt.ylabel("count")
plt.show()
```

```
stroke
0    4861
1     249
Name: count, dtype: int64
stroke
0    0.951272
1    0.048728
Name: proportion, dtype: float64
stroke
0    95.13
1     4.87
Name: proportion, dtype: float64

target distribution (stroke)
```



#mean, Median, Std

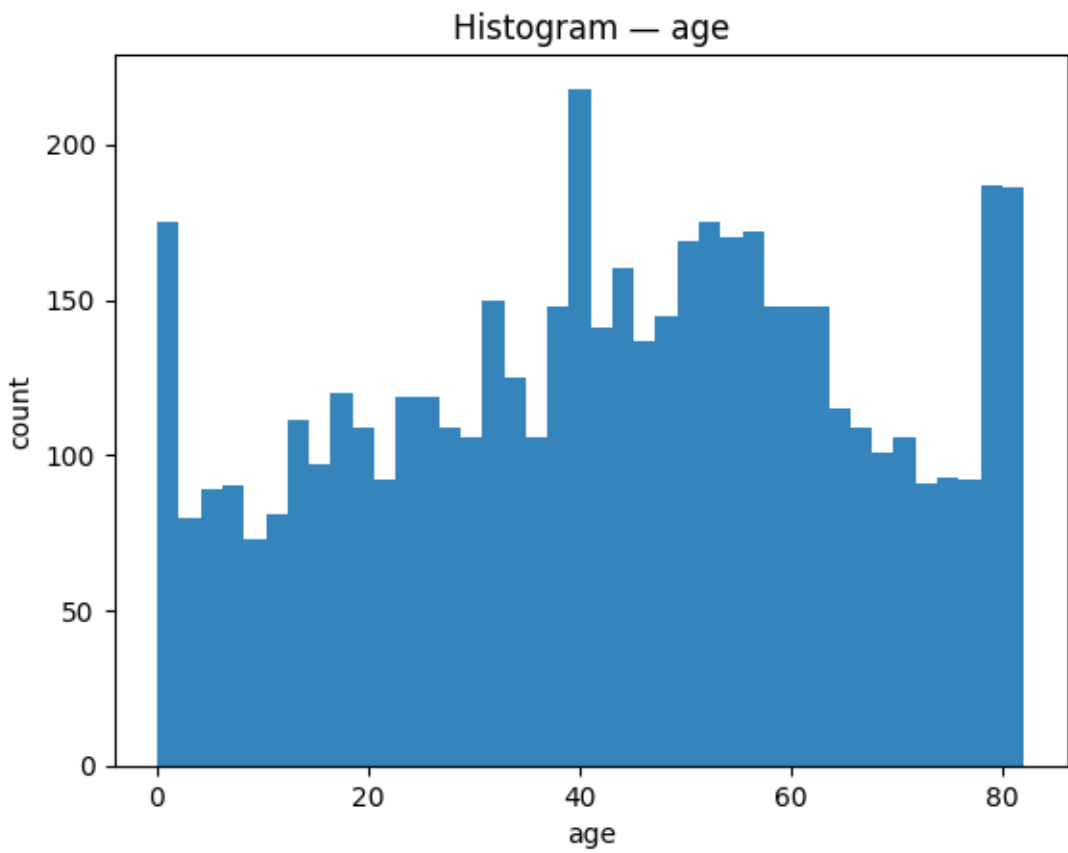
```
numeric_cols=["age", "avg_glucose_level","bmi"]
basic_stats=df[numeric_cols].agg(['mean','median','std','min','max']).T.round(3)
display(basic_stats)
```

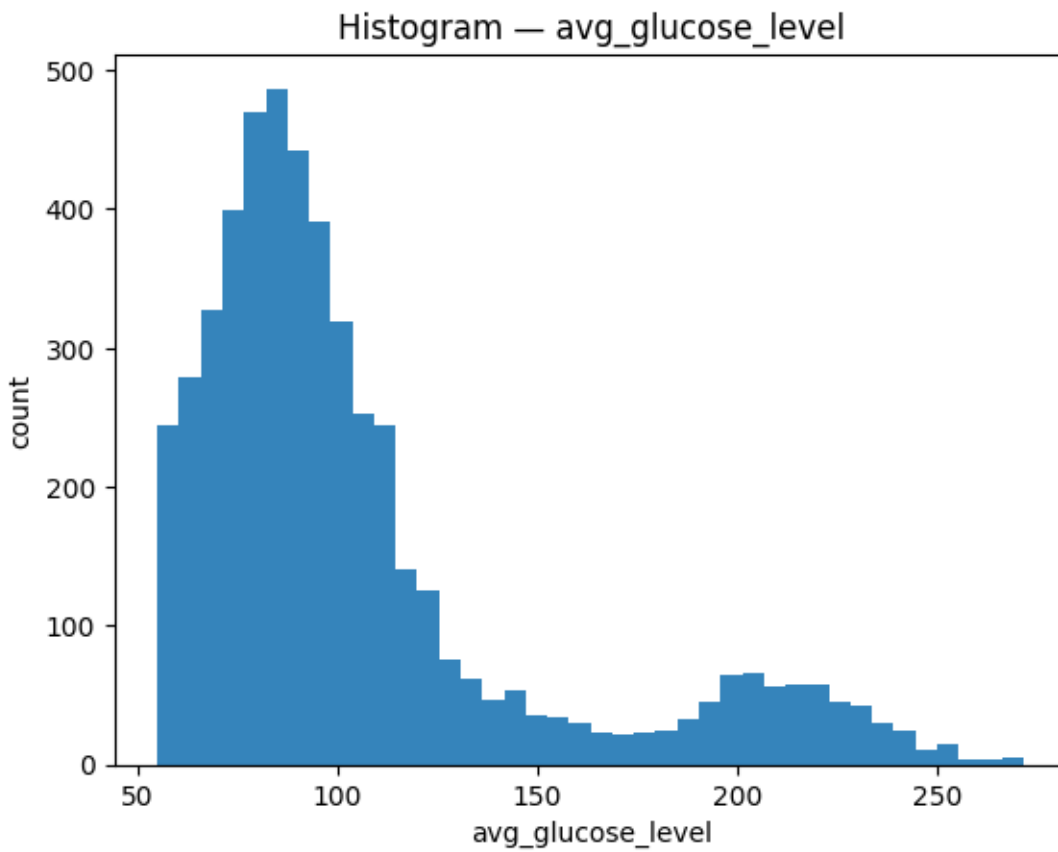
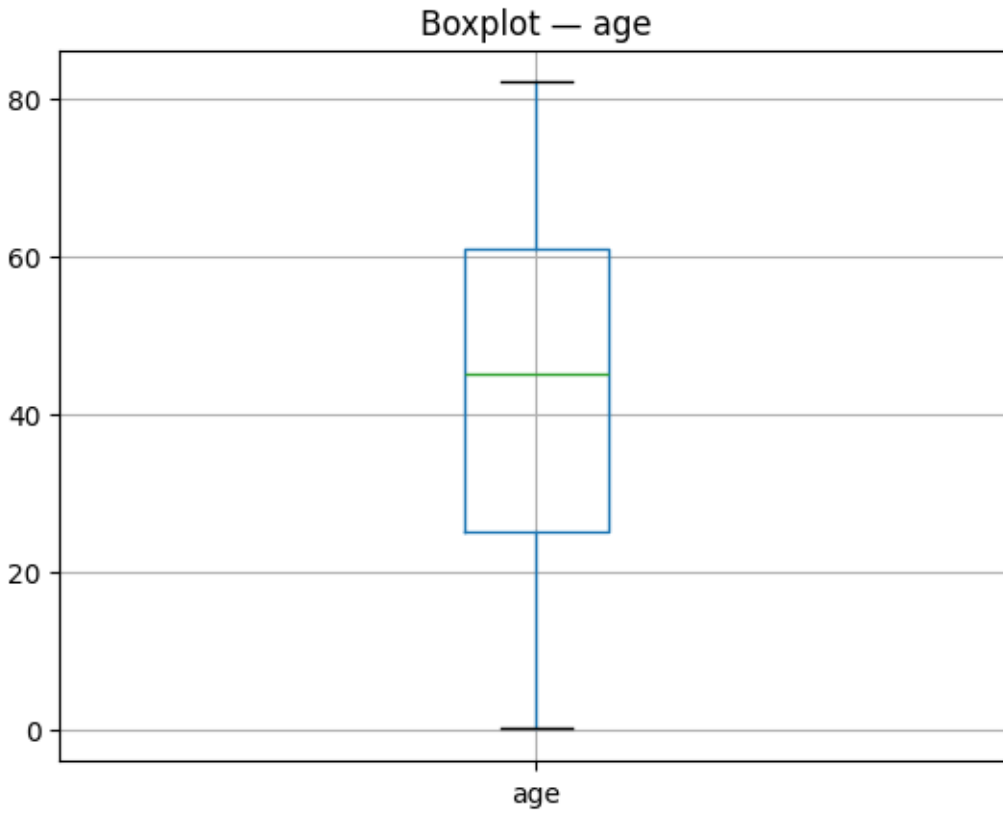
#Histogram & boxplot

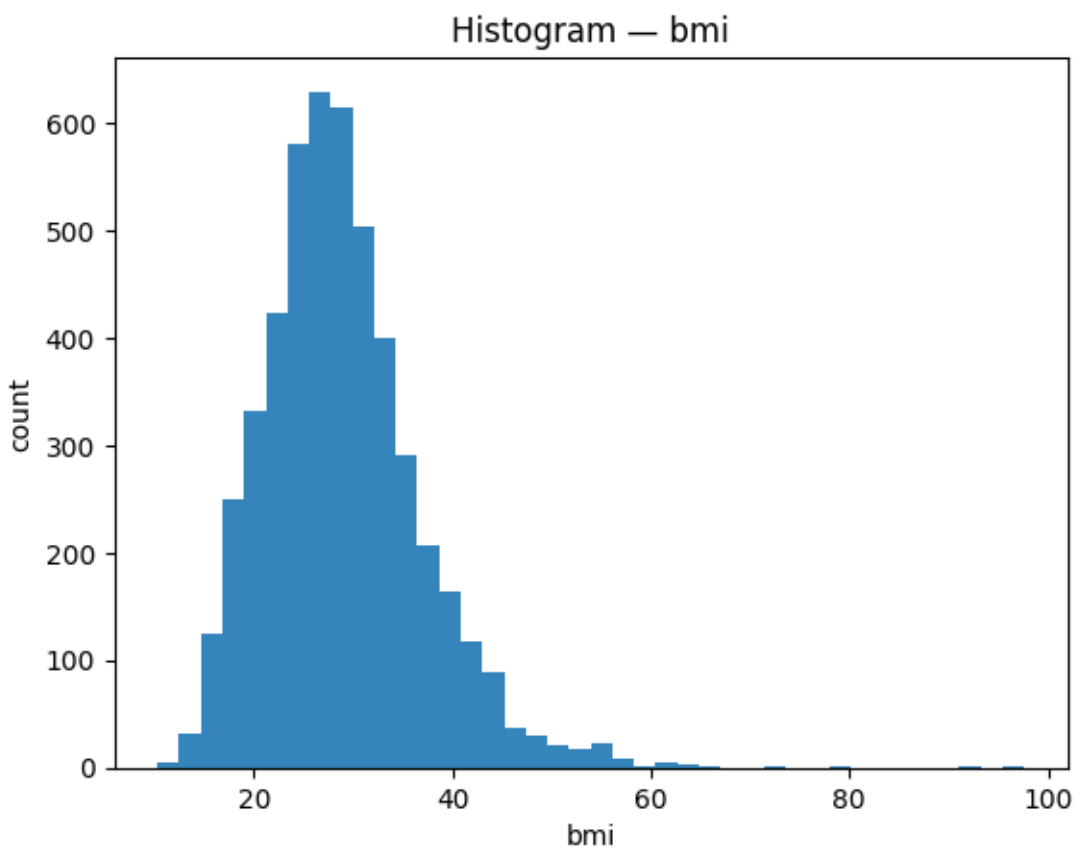
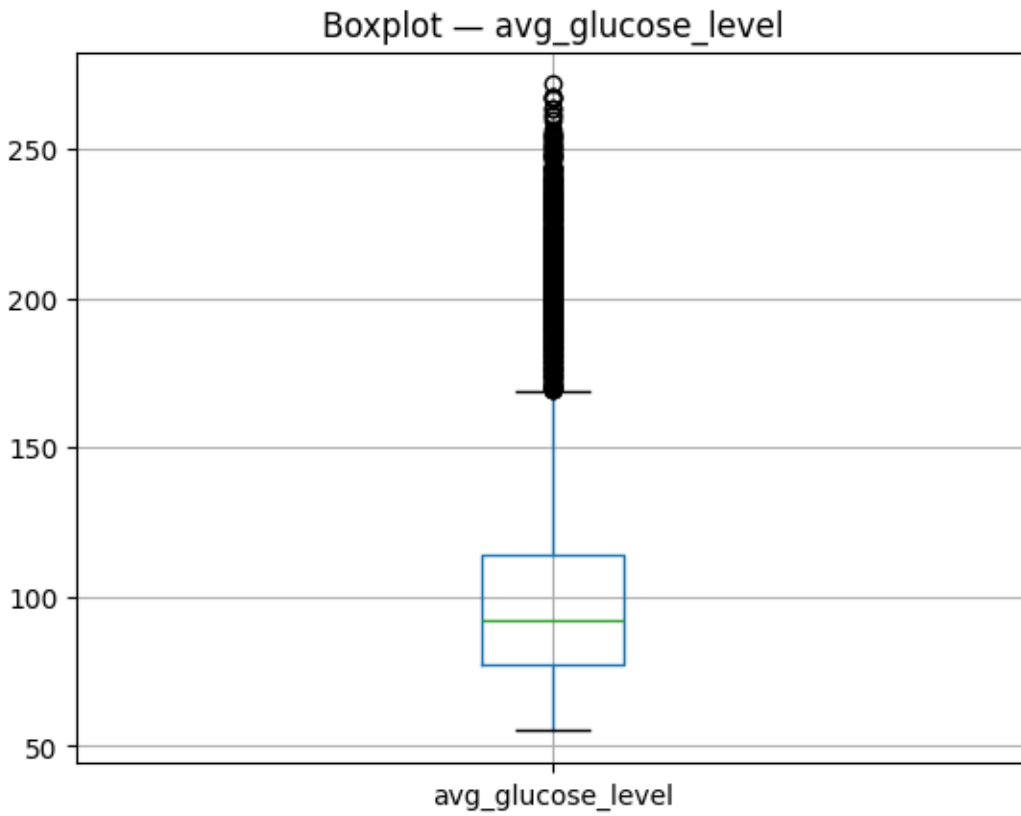
```
for a in numeric_cols:
    df[a].plot(kind='hist', bins=40, alpha=0.9,title=f'Histogram - {a}')
    plt.xlabel(a);
    plt.ylabel('count');
    plt.show()

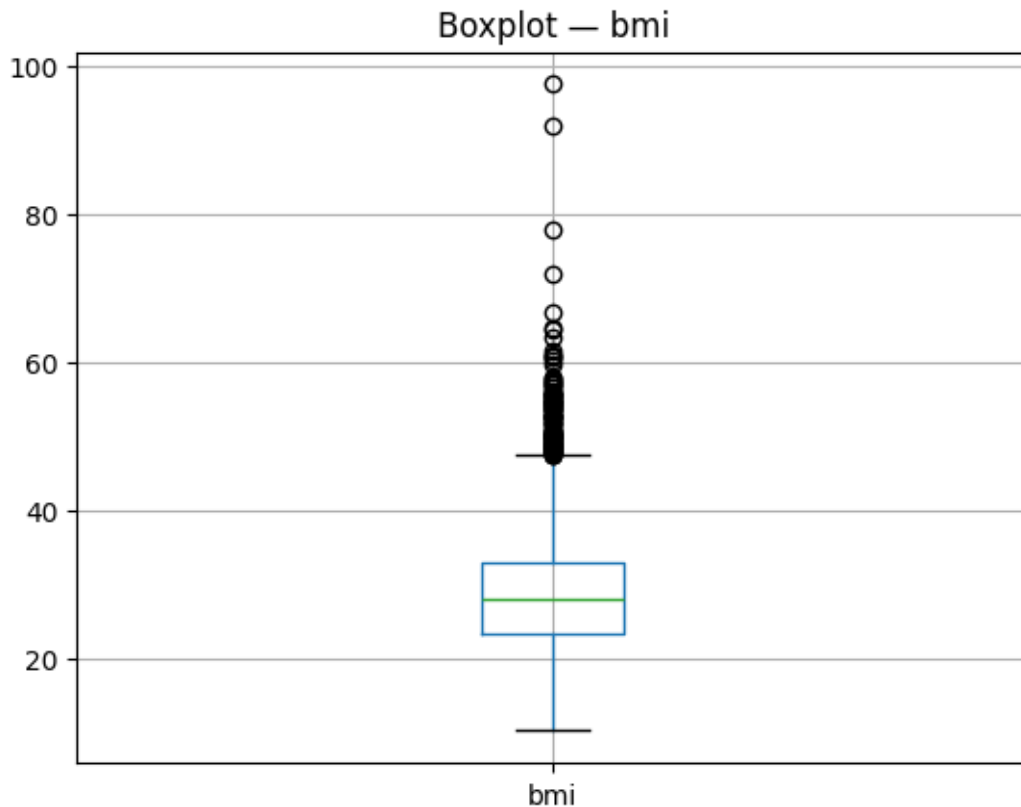
    df[[a]].boxplot(); plt.title(f'Boxplot - {a}');
    plt.show()
```

	mean	median	std	min	max	
age	43.227	45.000	22.613	0.08	82.00	
avg_glucose_level	106.148	91.885	45.284	55.12	271.74	
bmi	28.893	28.100	7.854	10.30	97.60	









```
correlation_target=df[numeric_cols+["stroke"]].corr(numeric_only=True).loc[numeric_cols,"stroke"].to_frame("pearson_with_target").round(3)
print(correlation_target)

correlation_matrix=df[numeric_cols].corr(numeric_only=True).round(3)
print(correlation_matrix)

#redundancy
pairwise= df[numeric_cols].corr().round(3)
display(pairwise)

pairwise_abs = pairwise.abs()
np.fill_diagonal(pairwise_abs.values, 0)
max_values = pairwise_abs.values.max()
i, j = np.where(pairwise_abs.values == max_values)
print(f"Strongest numeric pair: {pairwise.index[i[0]]} -> {pairwise.columns[j[0]]} | correlation = {pairwise.values[i[0], j[0]]:.3f}")

threshold= 0.6
pairwise=[]
for r in range(len(pairwise)):
    for c in range(r+1, len(pairwise)):
        correlation_values = pairwise.values[r, c]
        if abs(correlation_values) >= threshold:
            pairwise.append(pairwise.index[r],pairwise.columns[c], correlation_values)
print("strongly correlated pairs", pairwise or "None")
```

	pearson_with_target		
age			0.245
avg_glucose_level			0.132
bmi			0.042
	age	avg_glucose_level	bmi
age	1.000	0.238	0.333
avg_glucose_level	0.238	1.000	0.176
bmi	0.333	0.176	1.000
	age	avg_glucose_level	bmi
age	1.000	0.238	0.333
avg_glucose_level	0.238	1.000	0.176
bmi	0.333	0.176	1.000

Strongest numeric pair: age -> bmi | correlation = 0.333
 strongly correlated pairs None

CHAPTER TWO

Data Preparation/ Model Training/ Model Evaluation and Visualization

Data Preparation:

There is not duplicated, only missing values can be seen in BMI, about 4% in row201.

Once the numeric columns were imputed using the mean and median and the categorical ones with the most common value, all the columns had reached 0% missing.

I cleaned and X to all columns except id and stroke and y to stroke. One-Hot Encoding was used to deal with categorical features whereas StandardScaler and SimpleImputer worked with numeric features. The feature space was expanded to 21 to validate the new encoded features.

Leakage prevention: Only fit transform was run on the training set and transform on the test set, assertions were then made that no missing values were left in the final matrices.

Split and class ratio: I split using 80 to 20 train and test split stratified using stratify=y (Train: 4088, Test: 1022) where the original class skews were maintained in the two groups.

The preprocessing pipeline can be easily connected to models such as Logistic Regression, SVM, Decision Tree, Random Forest, KNN. The data are in standard form and complete, and thus we can go to training and use class-imbalance techniques like class weight -balanced or oversampling.

```
) #handling missing data  
df.isnull().sum()
```

```
;
```

	0
id	0
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	201
smoking_status	0
stroke	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
# fill missing values with mean column values
# Select only numeric columns for mean calculation
```

```
df2 = df.copy()
import warnings
warnings.filterwarnings('ignore')

numeric_cols=df2.select_dtypes(include=np.number)
df2[numeric_cols.columns]=df2[numeric_cols.columns].fillna(numeric_cols.mean())

# count the number of NaN values in each column

print(df2.isnull().sum())
```

df2

```
heart_disease    0
ever_married     0
work_type        0
Residence_type   0
avg_glucose_level 0
bmi              0
smoking_status   0
stroke           0
dtype: int64
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.600000	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	28.893237	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.500000	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.400000	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.000000	never smoked	1
...	...	...	...	...	...	...	...	...	...	...	...	...
5105	18234	Female	80.0	1	0	Yes	Private	Urban	83.75	28.893237	never smoked	0
5106	44873	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.000000	never smoked	0
5107	19723	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.600000	never smoked	0
5108	37544	Male	51.0	0	0	Yes	Private	Rural	166.29	25.600000	formerly smoked	0
5109	44679	Female	44.0	0	0	Yes	Govt_Job	Urban	85.28	26.200000	Unknown	0

5110 rows x 12 columns

```
#percentage
na_percentage= (df2.isnull().mean()*100).round(2)
df2.isnull().sum(axis=1)
missing_table=(pd.concat([na_count.rename('n_missing'),na_percentage.rename('percentage_missing')],axis=1).sort_values('n_missing',ascending=False))
print(missing_table)
```

	n_missing	percentage_missing
id	0	0.0
gender	0	0.0
age	0	0.0
hypertension	0	0.0
heart_disease	0	0.0
ever_married	0	0.0
work_type	0	0.0
Residence_type	0	0.0
avg_glucose_level	0	0.0
bmi	0	0.0
smoking_status	0	0.0
stroke	0	0.0

```
#howmany rows and columns there are in dataset
```

```
df.shape
print(df2.columns)
```

```
Index(['id', 'gender', 'age', 'hypertension', 'heart_disease', 'ever_married',
       'work_type', 'Residence_type', 'avg_glucose_level', 'bmi',
       'smoking_status', 'stroke'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
```

```
#X & y
X=df2.drop(columns=["stroke","id"])
y=df2["stroke"].astype(int)
```

```
#identifying Columns
```

```
categorical_cols=X.select_dtypes(include="object").columns.tolist()
numeric_cols=X.select_dtypes(include=['int64', 'float64']).columns.tolist()
```

```
#making pyplines
```

```
numeric_pipe=Pipeline(
    steps=[
        ("imputer",SimpleImputer(strategy="median")),
        ("scaler",StandardScaler())])

categorical_pipe=Pipeline(
    steps=[
        ("imputer",SimpleImputer(strategy="most_frequent")),
        ("onehot",OneHotEncoder(handle_unknown="ignore", sparse_output=False))])
```

```

preprocessing=ColumnTransformer(
    transformers=[
        ("numeric",numeric_pipe,numeric_cols),
        ("categorical",categorical_pipe,categorical_cols)])

#avoiding leakage
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42, stratify=y)

#fit & transform
X_train_processed=preprocessing.fit_transform(X_train) # fit only on train
X_test_processed=preprocessing.transform(X_test) # transform onky on test

print("train before and after:", X_train.shape, X_train_processed.shape)
print("test before and after:", X_test.shape, X_test_processed.shape)

#def no_missing_values(array):
#    if sparse.issparse(array):
#        return not np.isnan(array.data).any()
#    else:
#        return not np.isnan(array).any()

assert not np.isnan(X_train_processed).any()
assert not np.isnan(X_test_processed).any()
print("no missing values")

```

```

· train before and after: (4088, 10) (4088, 21)
  test before and after: (1022, 10) (1022, 21)
  no missing values

```

```

import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

#X & y
X=df2.drop(columns=["stroke","id"])
y=df2["stroke"].astype(int)

#splitting dataset into train and test
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42, stratify=y)

print(f"train:, X_train.shape, Positive (train): {int(y_train.sum())}")
print(f"test:, X_test.shape, Positive (test): {int(y_test.sum())}")
print("class ratio(train):", y_train.value_counts(normalize=True).sort_index().round(4).to_dict())
print("class ratio(test):", y_test.value_counts(normalize=True).sort_index().round(4).to_dict())
#models
models = {
    "LogReg": LogisticRegression(),
    "SVM-RBF": SVC(),
    "DecisionTree": DecisionTreeClassifier(),
    "RandomForest": RandomForestClassifier(),
    "KNN": KNeighborsClassifier()
}

#pipeline for each model
trained={}
for name, clf in models.items():
    pipe=Pipeline(steps=[("preprocessing",preprocessing),("classifier",clf)])
    pipe.fit(X_train,y_train)
    trained[name]=pipe

print(f"[{name}]fitted")

#hyperparameters for each model
def pick_params(clf, keys):
    p = clf.get_params()
    return {k: p.get(k, None) for k in keys}
params_report = {
    "LogReg": pick_params(models["LogReg"], ["penalty","c","solver","max_iter","class_weight"]),
    "SVM-RBF": pick_params(models["SVM-RBF"], ["kernel","c","gamma","degree","probability","class_weight"]),
    "DecisionTree":pick_params(models["DecisionTree"],["criterion","max_depth","min_samples_split","min_samples_leaf","max_features"]),
    "RandomForest":pick_params(models["RandomForest"],["n_estimators","criterion","max_depth","min_samples_split","min_samples_leaf","max_features"]),
    "KNN": pick_params(models["KNN"], ["n_neighbors","weights","metric"])
}

train:, X_train.shape, Positive (train): 199
test:, X_test.shape, Positive (test): 50
class ratio(train): {0: 0.9513, 1: 0.0487}
class ratio(test): {0: 0.9511, 1: 0.0489}
[KNN]fitted

```

	penalty	C	solver	max_iter	class_weight	kernel	gamma	degree	probability	criterion	max_depth	min_samples_split	min_samples_leaf	max_features	n_estimators
LogReg	l2	1.0	lbfgs	100	None	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
SVM-RBF	NaN	1.0	NaN	NaN	None	rbf	scale	3	False	NaN	NaN	NaN	NaN	NaN	NaN
DecisionTree	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	gini	None	2	1	None	NaN
RandomForest	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	gini	None	2	1	sqrt	100
KNN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```

import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, ConfusionMatrixDisplay, RocCurveDisplay)

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

models = {
    "LogReg": LogisticRegression(),
    "SVM-RBF": SVC(probability=True),
    "DecisionTree": DecisionTreeClassifier(),
    "RandomForest": RandomForestClassifier(),
    "KNN": KNeighborsClassifier()
}
pipes = {}
rows = []
for name, clf in models.items():
    pipe = Pipeline([("preprocessing", preprocessing), ("classifier", clf)])
    pipe.fit(X_train, y_train)
    pipes[name] = pipe
    y_pred = pipe.predict(X_test)
    if hasattr(pipe.named_steps["classifier"], "predict_proba"):
        y_score = pipe.predict_proba(X_test)[:, 1]
    elif hasattr(pipe.named_steps["classifier"], "decision_function"):
        y_score = pipe.decision_function(X_test)
    else:
        y_score = None

    rows.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1": f1_score(y_test, y_pred, zero_division=0),
        "ROC AUC": roc_auc_score(y_test, y_score).round(4) if y_score is not None else None,
    })

df_metrics = pd.DataFrame(rows).sort_values(by=["ROC AUC", "Recall", "F1"],
                                             ascending=False).round(4)

```

There were 5 models that were fitted in one pipeline and shared preprocessing; default hyperparameters were employed.

Splitting Imbalance: Train 4088, Test 1022 and a percentage of positives of about 5 and negative of about 95 in both sets- this has a strong effect on the metrics.

Test metrics table:

Most models have an accuracy of ≈ 0.95 (tree = 0.91), though this high accuracy is a demonstration of the majority class.

Most models have a recall of 0.02; nearly all of the positives are missed. The Decision Tree is the only that reaches Recall = 0.22, but with AUC = 0.58 and lower accuracy.

Precision LogReg gives a value of 1.0 since it makes only a single positive prediction, which was a correct prediction, the F1 remains very small (approximately 0.04.).

ROC-AUC has the best ranking, LogReg = 0.84 (good ranking ability) followed by the Random Forest = 0.79, the rest are smaller.

Confusion matrices:

LogReg: TN=972, FP=0, FN=49, TP=1 almost all positives were negatively predicted with default 0.5 level kills the recall.

SVM-RBF: TP=0, FN=50 that predicts the majority class on all cases.

ROC (LogReg): The curve is quite high above the diagonal (AUC ≈ 0.84), indicating that scores are informative, although by the threshold the binary classification does not find positives.

Overview: The default of most models is to the negative class due to the severe class imbalance. Although there are acceptable AUC of LogReg, Random Forest, Recall and F1 are quite low, and the confusion matrices indicate that positives are hardly detected.

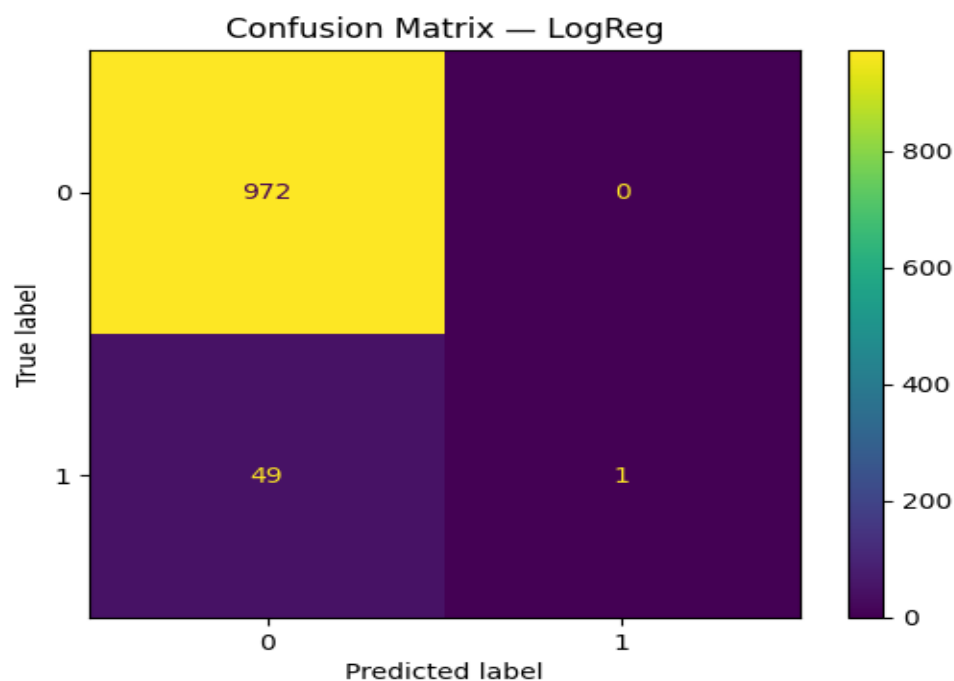
	Model	Accuracy	Precision	Recall	F1	ROC	AUC
0	LogReg	0.9521	1.0000	0.02	0.0392	0.8425	
3	RandomForest	0.9481	0.2000	0.02	0.0364	0.7928	
1	SVM-RBF	0.9511	0.0000	0.00	0.0000	0.6365	
4	KNN	0.9491	0.0000	0.00	0.0000	0.6208	
2	DecisionTree	0.9061	0.1618	0.22	0.1864	0.5807	

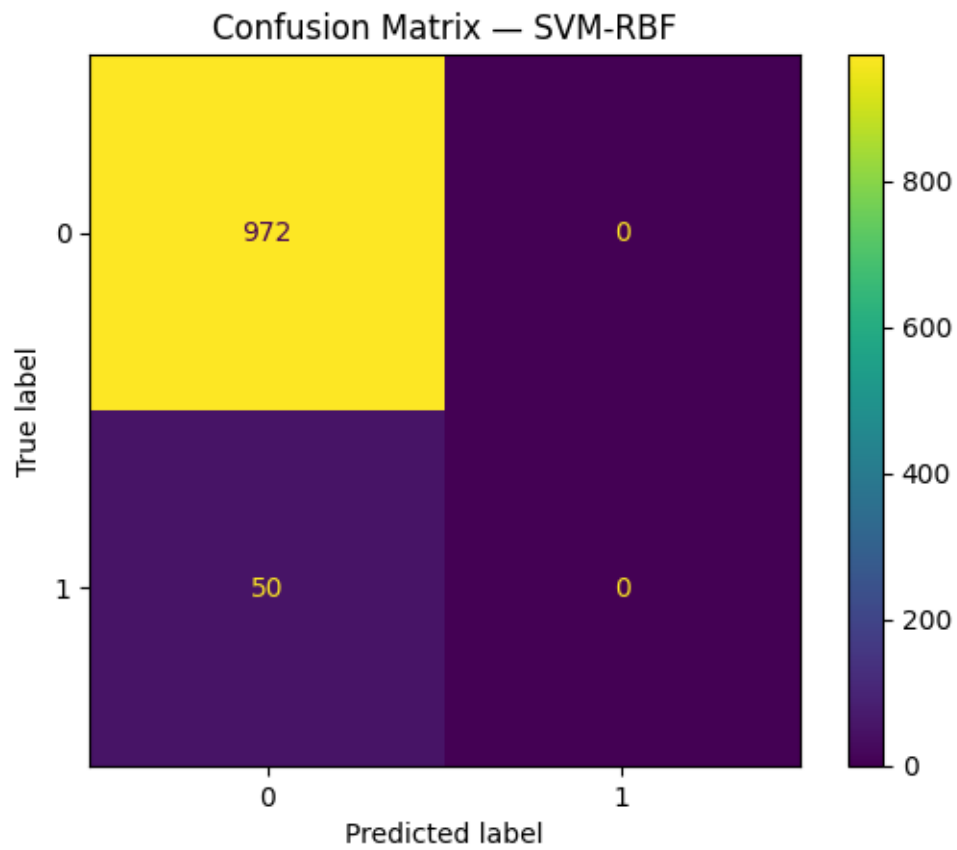
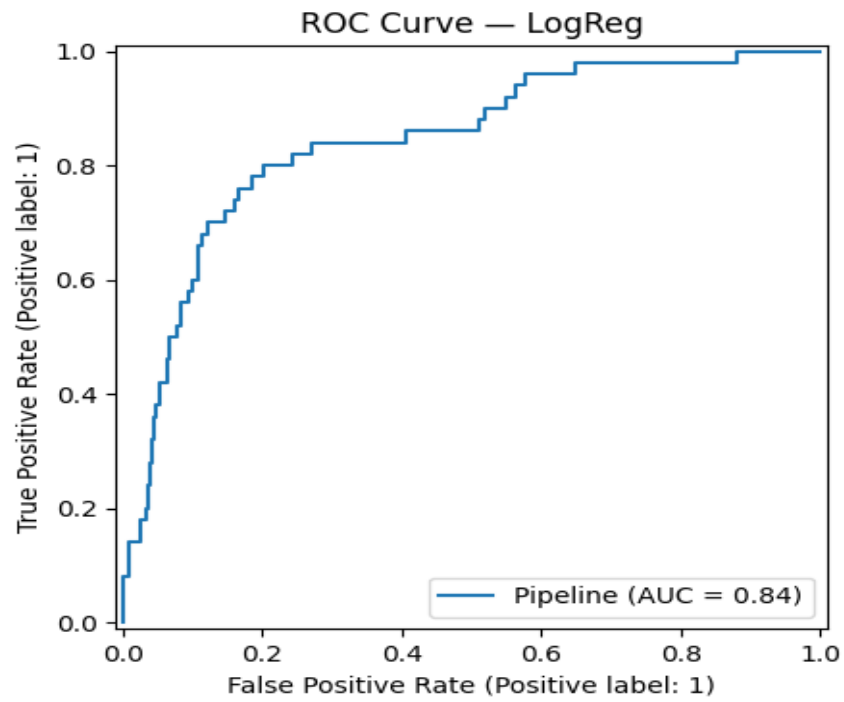
```

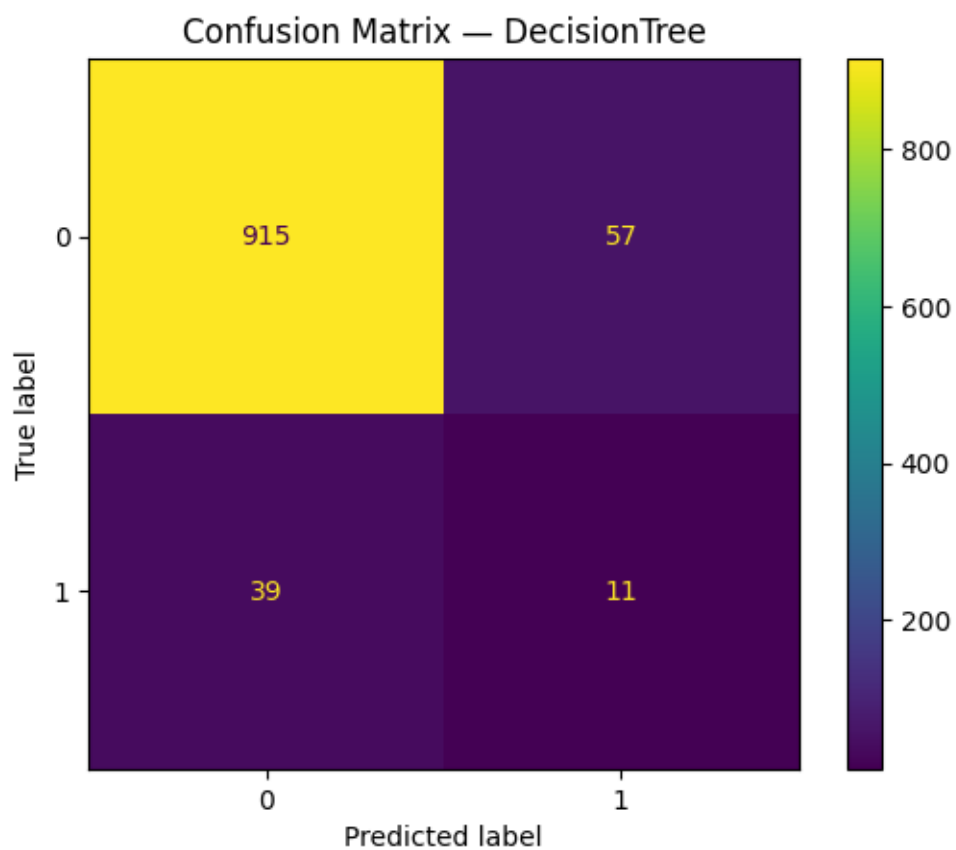
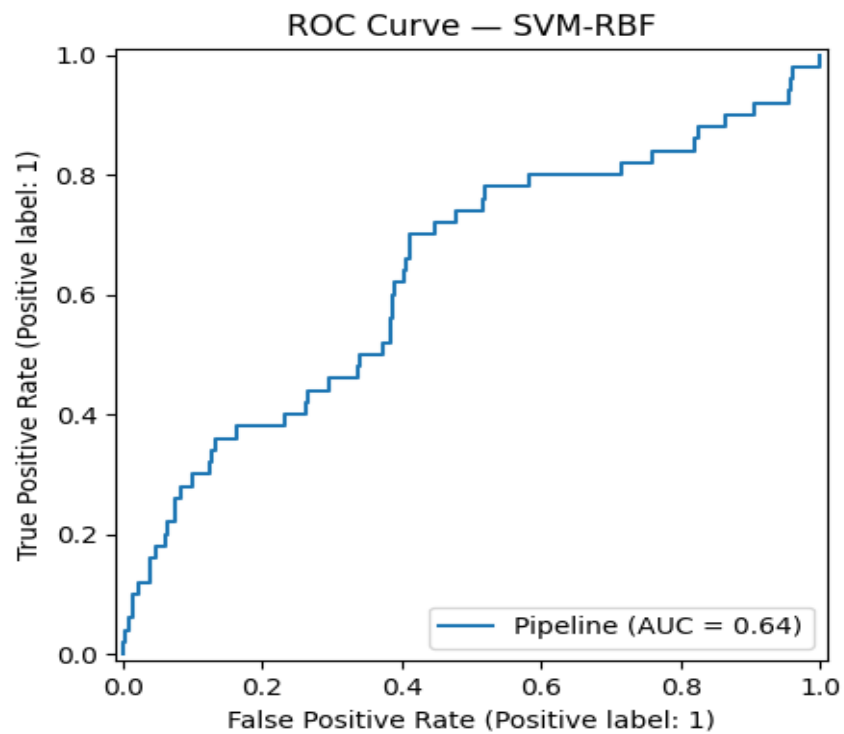
for name, pipe in pipes.items():
    plt.figure()
    ConfusionMatrixDisplay.from_estimator(pipe, X_test, y_test)
    plt.title(f"Confusion Matrix — {name}")
    plt.show()

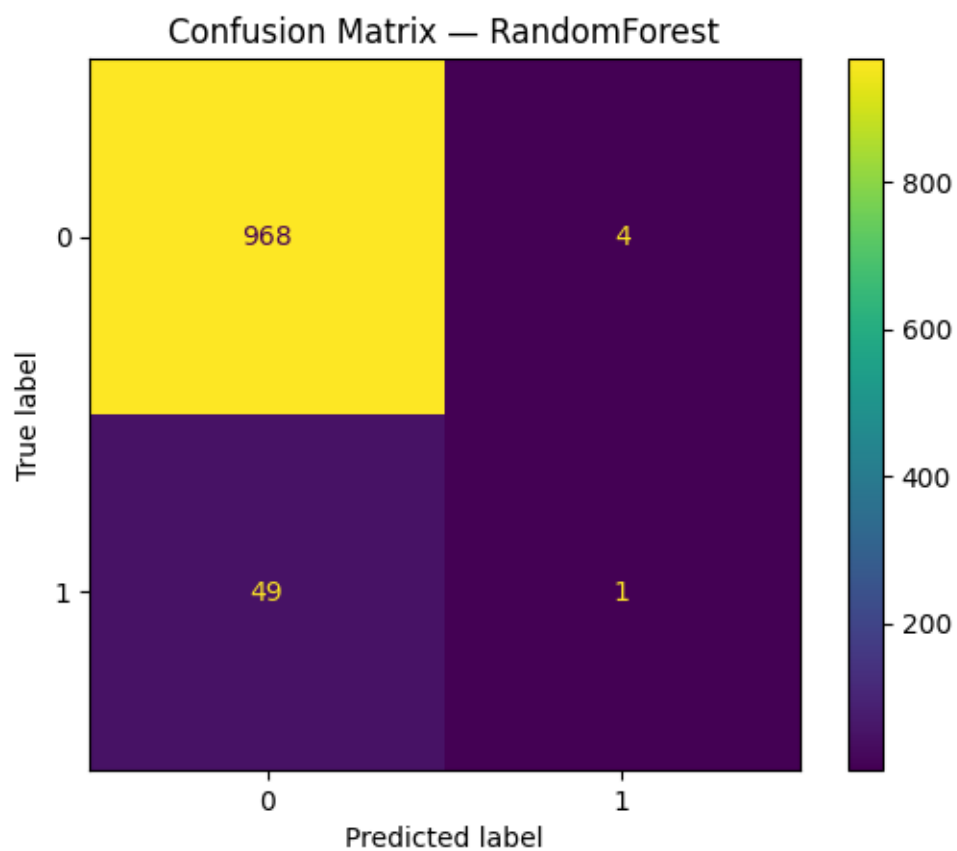
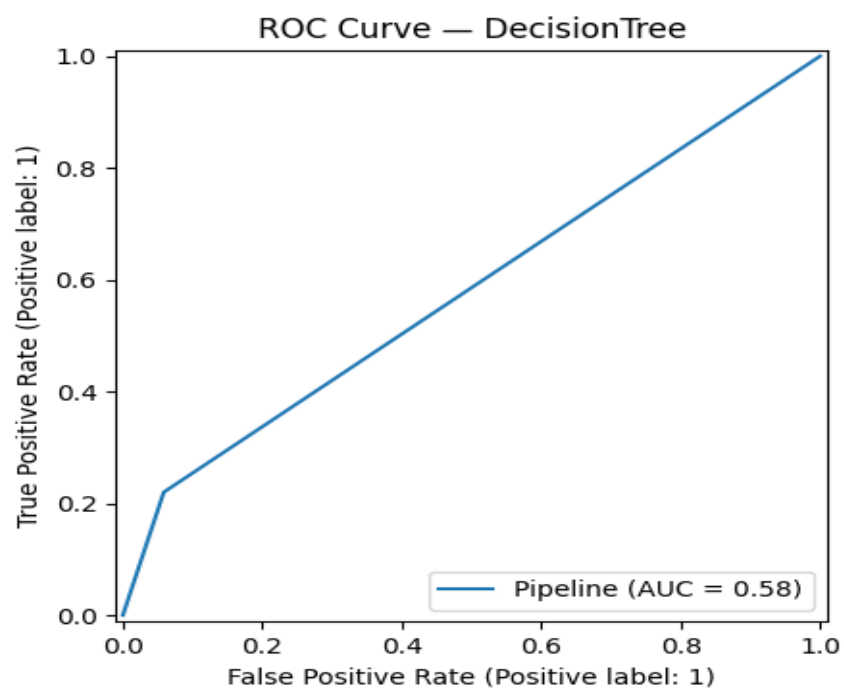
if hasattr(pipe.named_steps["classifier"], "predict_proba"):
    plt.figure()
    RocCurveDisplay.from_estimator(pipe, X_test, y_test)
    plt.title(f"ROC Curve — {name}")
    plt.show()

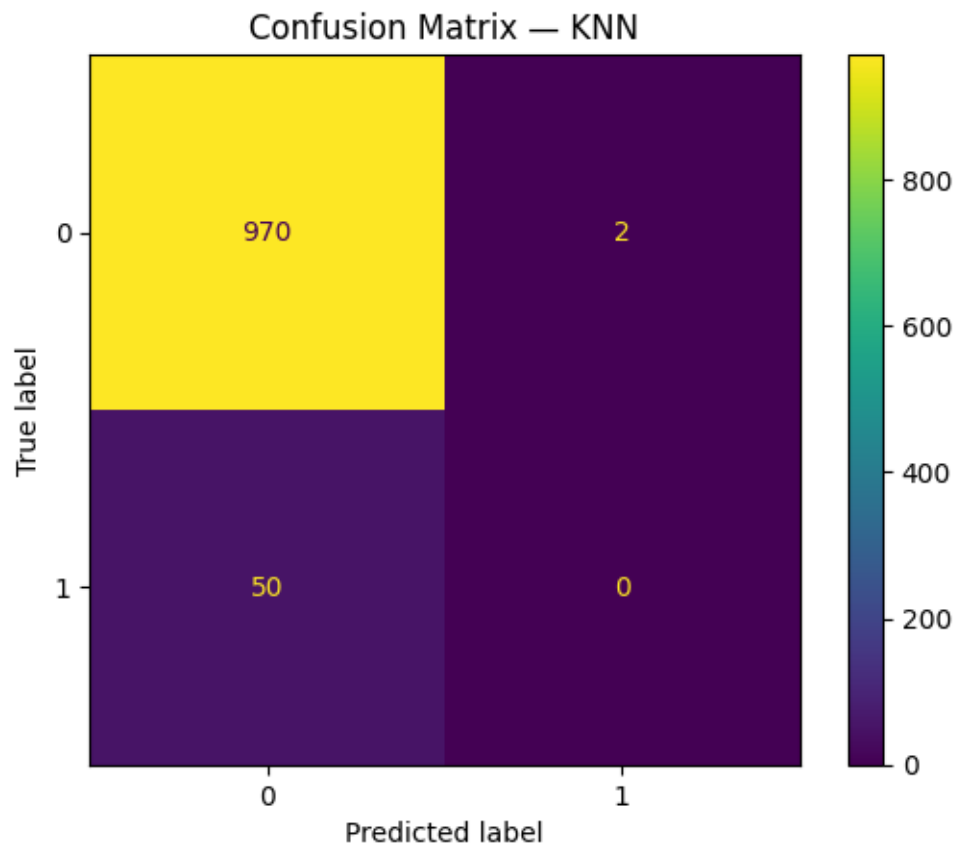
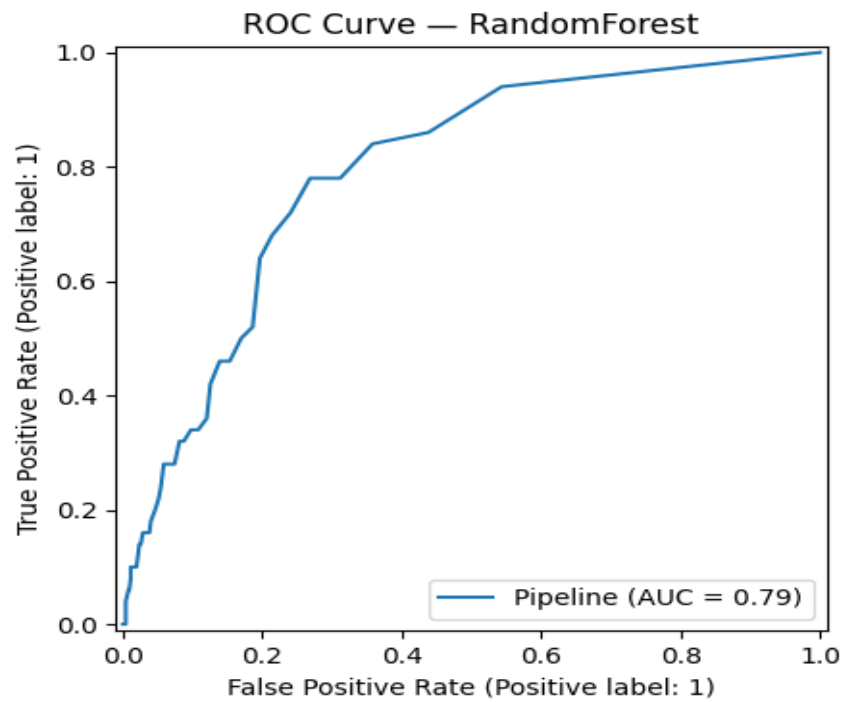
```

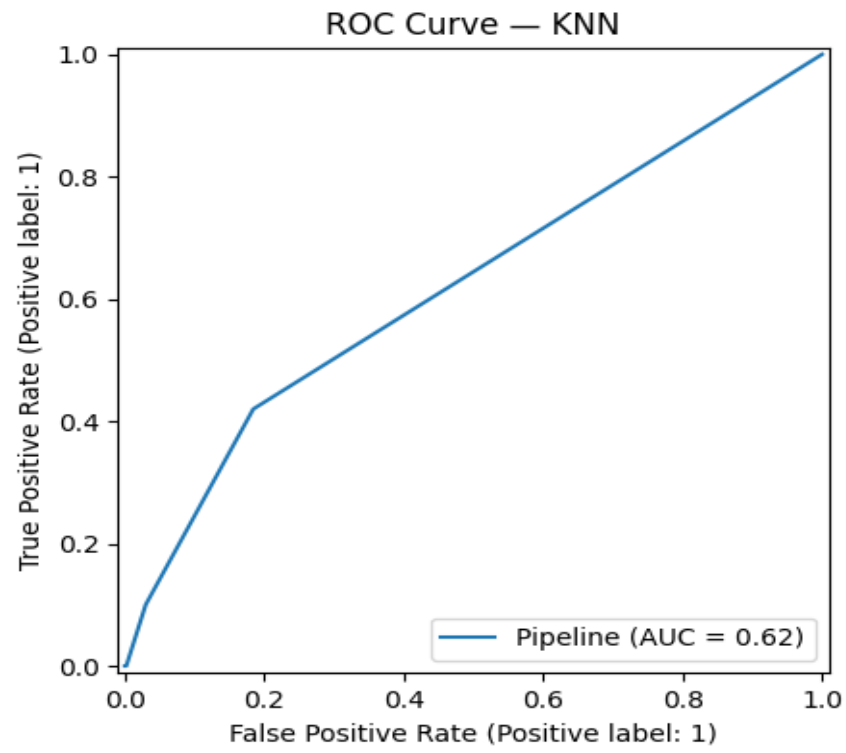












```

from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import classification_report
if df_metrics["ROC AUC"].isna().all():
    best_name=df_metrics.sort_values(by=["Recall", "F1", "Accuracy"], ascending=False).iloc[0]["Model"]

else:
    best_name=df_metrics.iloc[0]["Model"]

print("Best(pre-tuning):", best_name)

#training model
base_best=pipes[best_name]

|
def evaluation_metrics(model,label, clf_step_name):
    y_prediction=model.predict(X_test)
    if hasattr(model.named_steps[clf_step_name], "predict_proba"):
        y_score=model.predict_proba(X_test)[:,-1]
    elif hasattr(model.named_steps[clf_step_name], "decision_function"):
        y_score=model.decision_function(X_test)
    else:
        y_score=None
    row={
        "Label":label,
        "Accuracy":accuracy_score(y_test,y_prediction),
        "Precision":precision_score(y_test,y_prediction,zero_division=0),
        "Recall":recall_score(y_test,y_prediction,zero_division=0),
        "F1":f1_score(y_test,y_prediction,zero_division=0),
        "ROC AUC":roc_auc_score(y_test,y_score).round(4) if y_score is not None else None,}
    return row

pre_row=evaluation_metrics(base_best,"pre", "classifier")
if best_name == "RandomForest":
    base_estimator=RandomForestClassifier()
    param_grid={
        "clf__n_estimators":[200,400,600],
        "clf__max_depth":[None,10,20],
        "clf__min_samples_split":[2,5],
        "clf__min_samples_leaf":[1,2]
    }

elif best_name== "LogReg":
    base_estimator=LogisticRegression()
    param_grid={
        "clf__C":[0.1, 1,5,10],
        "clf__penalty":["l2"],
        "clf__solver":["lbfgs", "liblinear"],
        "clf__max_iter":[100,1000]}

```

```

elif best_name== "SVM-RBF":
    base_estimator=SVC(probability=True)
    param_grid={
        "clf__c":[0.5, 1,5],
        "clf__gamma":["scale",0.1,0.01]}

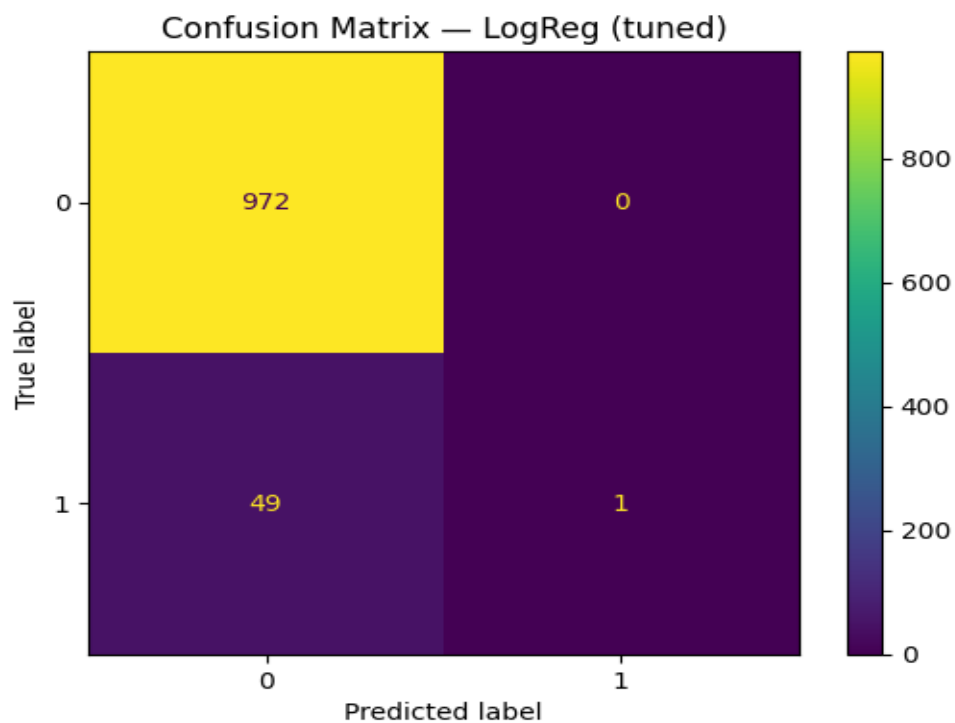
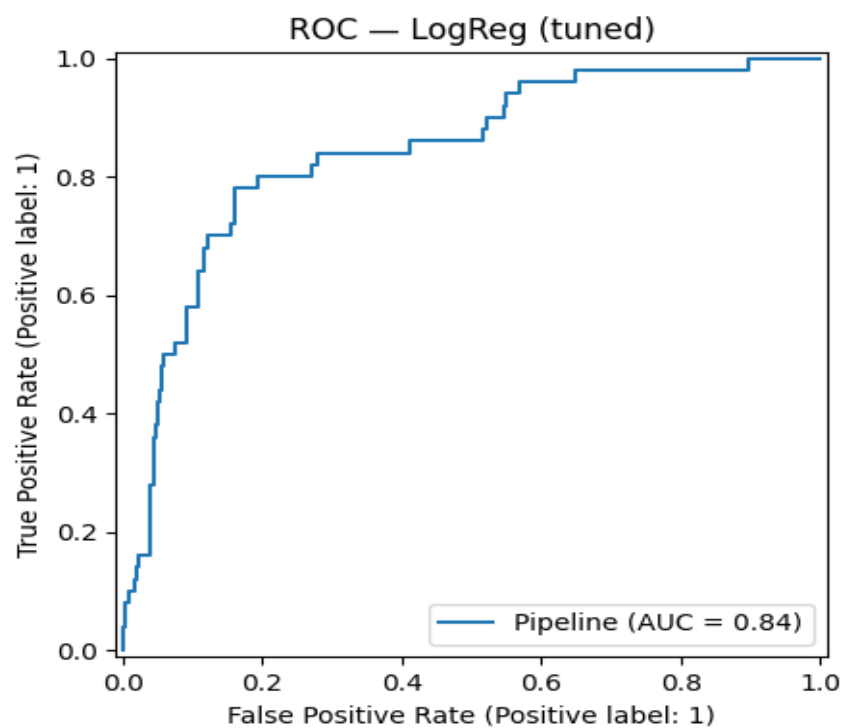
elif best_name== "DecisionTree":
    base_estimator=DecisionTreeClassifier()
    param_grid={
        "clf__max_depth":[None, 6 ,12,18],
        "clf__min_samples_split":[2,5,10],
        "clf__min_samples_leaf":[1,2,4] }
    #KNN
else:

    base_estimator=KNeighborsClassifier()
    param_grid={
        "clf__n_neighbors":[3,5,7,11,15, 21],
        "clf__weights":["uniform","distance"],
        "clf__p":[1,2]}

tune_pipeline= Pipeline([("pre", preprocessing),("clf",base_estimator)])
grid = GridSearchCV(tune_pipeline, param_grid, scoring="roc_auc", cv= 5, n_jobs=-1, verbose=0)
grid.fit(X_train, y_train)
print("Best params:", grid.best_params_)

```

	Label	Accuracy	Precision	Recall	F1	ROC AUC
0	pre	0.952	1.0	0.02	0.039	0.842
1	LogReg (tuned)	0.952	1.0	0.02	0.039	0.842



```

from sklearn.utils.multiclass import type_of_target
from sklearn.inspection import permutation_importance

type_y = type_of_target(y_test)

if type_y in ("binary",):
    candidates_scoring = ["roc_auc", "accuracy"]
elif type_y in ("multiclass", "multiclassovr"):
    candidates_scoring = ["roc_auc_ovr", "accuracy"]
else:
    candidates_scoring = ["r2", "neg_mean_squared_error"]

last_scoring_error = None
result = None
scoring_used = None

for sc in candidates_scoring:
    try:
        result = permutation_importance(
            clf, X_test_tr, y_test, n_repeats=10, random_state=42, scoring=sc )

        scoring_used = sc
        break
    except Exception as e:
        last_scoring_error = e
        result = None
        continue

if result is None:
    try:
        result = permutation_importance(
            clf, X_test_tr, y_test,
            n_repeats=10, random_state=42, scoring=None)

        scoring_used = "estimator.score"
    except Exception as e:
        raise RuntimeError(
            f"Could not compute permutation={clf.__class__.__name__}, "
            f"type_y={type_y}, tried={candidates_scoring + ['estimator.score']}. "
            f"Last error: {e if e else last_scoring_error}")

```

ROC — Logistic Regression (tuned): The curve lies on top of the diagonal with AUC = 0.84; there is no appreciable difference between the baseline and the tuned model-score ranking is good however.

Age, 0.29, that is the most influential feature. Average level glucose helpful to a small degree, hypertension and heart disease helpful to a small degree, bmi essentially non-helpful or is slightly negative.

The vast majority of categorical variables such as gender, work_type, smoking_status, residence_type, ever_married also have a near zero or even negative effect, which implies that they do not significantly enhance AUC in this model.

Overview: Tuned logistic model is dependent mainly on age other characteristics are insignificant. It is ranked ability good with high AUC, but the incremental information given by non-age variables is limited.

```
n_feats = len(result.importances_mean)
if len(feature_names) != n_feats:
    print(f" Warning: feature_names length ({len(feature_names)}) does not match importances length ({n_feats}) "
          f"(this usually happens when preprocessing adds or drops columns; truncating names to match the transformed matrix).")

imp = (
    pd.DataFrame({
        "feature": feature_names[:n_feats],
        f"importance_mean({scoring_used})": result.importances_mean,
        "importance_std": result.importances_std})

    .sort_values(by=f"importance_mean({scoring_used})", ascending=False)
    .reset_index(drop=True)
    .round(6))

display(imp.head(20))
```

```
from sklearn.utils.multiclass import type_of_target
from sklearn.inspection import permutation_importance

type_y = type_of_target(y_test)

if type_y in ("binary",):
    candidates_scoring = ["roc_auc", "accuracy"]
elif type_y in ("multiclass", "multiclassovr"):
    candidates_scoring = ["roc_auc_ovr", "accuracy"]
else:
    candidates_scoring = ["r2", "neg_mean_squared_error"]

last_scoring_error = None
result = None
scoring_used = None

for sc in candidates_scoring:
    try:
        result = permutation_importance(
            clf, X_test_tr, y_test, n_repeats=10, random_state=42, scoring=sc )
```

```

        scoring_used = sc
        break
    except Exception as e:
        last_scoring_error = e
        result = None
        continue

if result is None:
    try:
        result = permutation_importance(
            clf, X_test_tr, y_test,
            n_repeats=10, random_state=42, scoring=None)

        scoring_used = "estimator.score"
    except Exception as e:
        raise RuntimeError(
            f"Could not compute permutation={clf.__class__.__name__}, "
            f"type_y={type_y}, tried={candidates_scoring + ['estimator.score']}. "
            f"Last error: {e if e else last_scoring_error}")

n_feats = len(result.importances_mean)
if len(feature_names) != n_feats:
    print(f" Warning: feature_names length ({len(feature_names)}) does not match importances length ({n_feats}) "
          f"(this usually happens when preprocessing adds or drops columns; truncating names to match the transformed matrix).")

imp = (
    pd.DataFrame({
        "feature": feature_names[:n_feats],
        f"importance_mean({scoring_used})": result.importances_mean,
        "importance_std": result.importances_std})

    .sort_values(by=f"importance_mean({scoring_used})", ascending=False)
    .reset_index(drop=True)
    .round(6))

display(imp.head(20))

```

	feature	importance_mean(roc_auc)	importance_std
0	numeric__age	0.292062	0.041207
1	numeric__avg_glucose_level	0.004360	0.002672
2	numeric__hypertension	0.001644	0.002058
3	numeric__heart_disease	0.001504	0.001505
4	categorical__smoking_status_never smoked	0.001354	0.001022
5	categorical__work_type_Private	0.001095	0.000867
6	categorical__smoking_status_formerly smoked	0.000183	0.000609
7	categorical__work_type_Self-employed	0.000111	0.000485
8	categorical__gender_Male	0.000041	0.000215
9	categorical__gender_Female	0.000039	0.000201
10	categorical__gender_Other	0.000000	0.000000
11	categorical__work_type_Never_worked	-0.000002	0.000014
12	categorical__work_type_children	-0.000031	0.000085
13	numeric__bmi	-0.000058	0.000029
14	categorical__work_type_Govt_job	-0.000181	0.000217
15	categorical__ever_married_Yes	-0.000331	0.000635
16	categorical__ever_married_No	-0.000335	0.000632
17	categorical__Residence_type_Rural	-0.000346	0.000335
18	categorical__Residence_type_Urban	-0.000350	0.000340
19	categorical__smoking_status_Unknown	-0.000401	0.000244

CHAPTER THREE

Future Work

Summary of Findings:

Best model:

The overall discrimination on the test set was the highest when using the Logistic Regression (ROC-AUC ≈ 0.842 0.843 , Accuracy ≈ 0.952). Its operating point is however, very conservative (Precision = 1.00, Recall = 0.02, F1 = 0.039), meaning that it predicts very few positives. The tuned version basically did the same as the pre-tuned model and therefore no significant gains were realized by the existing tuning.

Key insights:

Permutation importance indicates a very strong concentrated signal: age is by far the most dominant predictor (Δ AUC 0.292 0.041), whereas avg_glucose level contributes a small, but significant signal (0.004), and other characteristics (hypertension, heart disease, work type, smoking status etc.) are not significant and are likely due to one-hot complements and class imbalance.

The model is reasonable in terms of aggregate (AUC) classification, but in the default 0.5 threshold, there are most positives that are missed (very low Recall). Threshold selection and imbalance control are both required in any application where false negatives are expensive.

Limitations

Class imbalance:

This may indicate that the minority class is being undetected at the default threshold.

feature representation:

One-hot, drop=first: Complementary columns are generated which dilute or mask importance; there are some categories which are exceptionally infrequent such as gender_Other.

Depending on one driver:

The high dependence on age makes issues of robustness when the age distribution of a target population changes.

External validity External or temporal validation:

The results are based on one dataset/population; external validity is necessary to make the claim of generalisability.

Future Work:

Decision threshold and metrics

- Optimize the decision threshold based on Youden J, cost-sensitive criteria or F1.
- Report PR-CAUC, precision and recall at the selected threshold, and cost-conscious measures, rather than ROC-AUC.

Imbalance handling:

Repeated stratified CV can prevent overfitting with use of class weight=balanced LogReg, and SVM or sampling algorithms (SMOTE and undersampling).

Feature engineering:

- One-hot enable drop = first: This encoding format collapses ultra-rare categories.
- Include clinically meaningful interactions for example, age x hypertension, age x avg_glucose level, mild non-linearities (age binning, and splines).

Models and tuning:

- Modern gradient boosting LightGBM, XGBoost, and CatBoost using Bayesian optimization Optuna or Skopt and, when suitable, monotonic constraints should be tried.
- Calibrate probabilities (Platt or isotonic) in order to have good thresholding.

Explainability and error analysis:

- Supplement permutation importance with SHAP (when using tree models) or PD and ICE plots to understand the effect of age and glucose on probability.
- Perform specific error analysis of false negatives in order to inform new features and threshold decisions.

Conclusion:

Logistic Regression is currently providing the most desirable discrimination (AUC) and it works at the point of unhelpfulness in terms of recall. The most obvious levers to increase real-world utility without reducing interpretability can be shifting the decision threshold, class imbalance, feature refinement and interaction refinement in particular and calibrated boosted models exploration.

CONCLUDING REMARKS

This project employed multiple classification models and machine learning models, including Decision Tree, Random Forest, and Support Vector Machine to predict stroke risks. Additionally, models were evaluated to identify the best-fit model for healthcare-dataset-stroke-data. Among numeric features, age shows the strongest link to stroke, while avg_glucose_level adds weaker but meaningful signal and has many high outliers. Model performance was evaluated using metrics such as Accuracy, Precision, F1 Score, and Recall. The results show that the best model is Logistics Regression in terms of ROC and AUC, but not in terms of practical performance (F1 or Recall), that is expected with unusual data where PR measures are more revealing ([Saito and Rehmsmeier, 2015](#)). While XGBoost and LightGBM have higher performance and they are able to capture non-linear interaction in tabular healthcare data. ([Chen and Guestrin, 2016](#)).

BIBLIOGRAPHY

1. NHS. (n.d.). *Stroke*. National Health Service. Available at: <https://www.nhs.uk/conditions/stroke/> [Accessed 30 Sep. 2025]
2. Daidone, M., Ferrantelli, S. and Tuttolomondo, A., 2023. Machine learning applications in stroke medicine: advancements, challenges, and future perspectives. *Neural Regeneration Research*, 19(4), pp.769-773. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10664112/> [Accessed 30 Sep. 2025]
3. Saito, T. and Rehmsmeier, M. (2015) the precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets, *Plos One*, 10(3), e0118432. Available at https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0118432&utm_source [Accessed 30 Sep. 2025]
4. Chen, T., Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 16)*. New York: Available at https://dl.acm.org/doi/10.1145/2939672.2939785?utm_source [Accessed 30 Sep 2025]

APPENDIX (if necessary)

